

Examen: Adopción de Mascotas con JavaScript

Objetivo del Examen

En este examen práctico deberás desarrollar una **plataforma interactiva de adopción de mascotas**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local** en **Node.js/Express**.

La aplicación debe permitir:

1. Consultar el catálogo de animales disponibles y las especies desde el servidor backend local (`/api/animales` y `/api/especies`).
2. Renderizar las tarjetas de los animales en `#catalogoGrid` con nombre, raza, edad, refugio, fotos HD y badges informativos, y poblar las opciones del selector `#filtroEspecie` .
3. Filtrar animales por especie (`#filtroEspecie`).
4. Abrir la ficha de adopción en `#modalDetalle` al hacer click en el botón "Conocer y Adoptar" de cualquier tarjeta, mostrando la información del animal y un formulario `#formAdopcion` para ingresar los datos del adoptante (`#inputNombreAdoptante` y `#inputTelefono`).
5. Persistir las solicitudes de adopción en `localStorage` bajo la clave `'adopcion_solicitudes'` , renderizarlas en `#solicitudesLista` y permitir limpiarlas con `#btnLimpiarSolicitudes` .

 **Tip para el examen:** En `index.html` encontrarás bloques de código HTML comentados con las plantillas exactas de la tarjeta (`.animal-card`), del modal (`.detail-header-info`) y de la solicitud guardada (`.solicitud-card`). Podés copiar y pegar estas plantillas directamente en tu código JavaScript (usando template literals con comillas invertidas ``) para no perder tiempo escribiendo estructura HTML ni clases CSS.

Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/animales</code> y <code>/api/especies</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de mascotas con fetch y async await</code>
#3	Renderizar las tarjetas de animales en <code>#catalogoGrid</code> y poblar el selector <code>#filtroEspecie</code> .	<code>feat(js): renderizar tarjetas de animales y filtros en el dom</code>
#4	Implementar filtrado por especie y modal de ficha de adopción.	<code>feat(js): implementar filtrado y ficha de adopcion</code>
#5	Persistir solicitudes en <code>localStorage</code> , mostrarlas en <code>#solicitudesLista</code> y limpiarlas con	<code>feat(js): persistir y gestionar solicitudes en localStorage</code>

Entrega	Tarea a Realizar	Commit Sugerido
	<code>#btnLimpiarSolicitudes</code> .	

Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/animales` : Devuelve la lista de animales disponibles.
- `GET http://localhost:3000/api/animales/:id` : Devuelve la ficha de un animal por su ID numérico.
- `GET http://localhost:3000/api/especies` : Devuelve las especies registradas (`id` , `nombre`).

Para iniciar el servidor backend:

```
npm start
```

2. Elementos Clave del DOM

- `#filtroEspecie` : `<select>` para filtrar por especie (Todas las especies, Perros, Gatos).
- `#catalogoGrid` : Contenedor grid donde se renderizan las tarjetas de animales (`.animal-card`).
- `#modalDetalle` : Modal emergente que contiene la ficha y el formulario de adopción.
- `#detalleContent` : Contenedor dentro del modal donde se renderiza la información detallada del animal.
- `#formAdopcion` : Formulario de solicitud de adopción.
- `#inputNombreAdoptante` : Campo de texto para el nombre del solicitante.
- `#inputTelefono` : Campo de teléfono del solicitante.
- `#btnConfirmarAdopcion` : Botón submit para registrar la solicitud.
- `#solicitudesLista` : Contenedor donde se listan las solicitudes enviadas.
- `#btnLimpiarSolicitudes` : Botón para vaciar el historial de solicitudes en `localStorage` .
- `.cerrar-modal` : Botones para cerrar el modal de adopción.

3. Lógica de Filtrado y Solicitudes

- Al cambiar el selector `#filtroEspecie` (`change`), se filtran las tarjetas mostrando solo los animales de esa especie.
- Al hacer click en el botón de adopción (`.btn-adoptar`) de una tarjeta, consultar `/api/animales/:id` (o buscar en el arreglo cargado) y mostrar la información en `#modalDetalle` .
- Al enviar `#formAdopcion` (`submit`), prevenir la recarga (`e.preventDefault()`), crear la solicitud, guardarla en `localStorage` , actualizar `#solicitudesLista` y cerrar el modal.

4. Almacenamiento Local (`localStorage`)

- Clave obligatoria: `'adopcion_solicitudes'`
- Estructura: Arreglo de objetos con `{ id, animalNombre, especie, solicitante, telefono, fecha }` .

- Utilizar `JSON.stringify()` para persistir y `JSON.parse()` para recuperar los datos.
- `#btnLimpiarSolicitudes` debe usar `localStorage.removeItem('adopcion_solicitudes')` o `localStorage.clear()`.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias: `bash npm install`
2. Iniciar el servidor local: `bash npm start`
3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).
4. Abrir la consola de herramientas de desarrollador (F12) para verificar peticiones de red y depurar posibles errores.